

Unit 1 — Introducing Streaming Data & Data Ingestion

Stream Processing and Analytics (AI372IA) · RVCE VII Sem · Syllabus: real-time systems · real-time vs streaming · architectural blueprint · security · scaling · common interaction patterns · scaling the patterns · fault tolerance

Sources used: Psaltis *Streaming Data* Ch.1–2 (printed pp.3–37) + faculty PPT "Unit I" + all CIE/SEE/Model papers. **BOOK p.xx** ↗ chips open *Streaming-Data[1].pdf* at that printed page (PDF page = printed + 20). Blue **highlight** = must-remember. **IMPORTANT** = asked in a past paper.

0 · Exam pattern & what Unit 1 looks like in the paper

Part	Pattern (from SEE Feb'26, Model QP, syllabus rubric)
Part A (Q1)	10 short questions × 2 marks = 20 marks , whole syllabus (all units). Some are fill-ups / scenario one-liners.
Part B	5 full questions × 16 marks = 80 . Q2 (Unit 1) is COMPULSORY . Then one of Q3/Q4 (Unit 2), Q5/Q6 (Unit 3), Q7/Q8 (Unit 4), Q9/Q10 (Unit 5). Each question has max 3 sub-parts (e.g. 8+8, 10+6).
Unit 1 in SEE Feb'26	Q2a (8) "streaming pattern for collecting data with an example"; Q2b (8) "publish/subscribe pattern with neat diagram". Part A: 1.1 stateless design, 1.2 which interaction pattern, 1.3 which logging technique.

Topic hot-list (ranked by how often asked)

Rank	Topic	Where it appeared	Notes section
1	Interaction patterns (esp. Pub/Sub , Stream, compare all)	SEE Feb'26 Q2a,Q2b,1.2 · CIE-1 Dec'24 Q2a,Q3a · CIE-1 Nov'25 PartA-1, Q2b	\$4
2	Streaming data architecture (draw + explain) for a scenario	Model QP Q2a · CIE-1 Dec'24 Q1a (vehicle tracking) · CIE-1 Nov'25 Q1a (IoT)	\$2, \$8
3	RBML / SBML / HML, checkpointing	CIE-1 Dec'24 Q3b · CIE-1 Nov'25 PartA-3, Q1a · SEE Feb'26 1.3 · Model QP 1.2	\$6
4	Real-time classes (hard/soft/near) & streaming vs real-time	Model QP 1.1 · CIE-1 Dec'24 PartA-1 · CIE-1 Nov'25 Q2a	\$1
5	Scaling (vertical vs horizontal; importance)	Model QP Q2b · CIE-1 Dec'24 PartA-3 · CIE-1 Nov'25 Q1a	\$3, \$5
6	Needs for streaming data; stateless design	CIE-1 Dec'24 & Nov'25 PartA-2 · SEE Feb'26 1.1	\$1

1 · Real-time systems vs streaming systems

1.1 What is a real-time system? [BOOK p.4 ↗](#)

- A real-time system produces results within a **time limit**. Three classes: **Hard, Soft, Near** (hard/soft after Kopetz; near from Portland Pattern wiki).
- **Hard**: very strict deadline; a miss ⇒ **total system failure / loss of life**. Embedded systems. (Out of scope of the book.)
- Soft vs near: **the line is blurry, subjective and depends on the consumer** of the data.

Table 1.1 (textbook) — Classification of real-time systems [BOOK p.5 ↗](#)

Class	Examples	Latency measured in	Tolerance for delay
Hard	Pacemaker, anti-lock brakes	Microseconds–milliseconds	None — system failure, loss of life
Soft	Airline reservation system, online stock quotes, VoIP (Skype)	Milliseconds–seconds	Low — no system failure, no life at risk
Near	Skype video, home automation	Seconds–minutes	High — no system failure, no life at risk

Faculty PPT slides 7 & 9 — extra comparison (hard vs soft; soft vs near)

Hard vs Soft: response time **hard = perform predictably** / soft = degraded · error detection: system / user · safety: critical / non-critical · redundancy: active / standby · time granularity: millisecond / second · data files: small-medium / large.

Soft vs Near: timing precision — soft "deadlines important but flexible", near "processed as quickly as possible with very minimal delay" · tolerance — soft "some delay or missed deadlines OK", near "some lag (often in seconds) acceptable" · examples — soft: video streaming, VoIP, online gaming; near: stock trading, traffic monitoring, sensor data processing.

⚠ **CONFLICT / TRAP — Soft vs Near examples differ between sources**

Textbook Table 1.1 puts *online stock quotes* under **soft** and *Skype video / home automation* under **near**, and says near has **higher** delay tolerance (seconds–minutes). The **faculty slides (slide 9/12) and the CIE-1 Nov'25 solution** put *stock trading* under **near** and *video streaming* under **soft**, and describe near as "very low delay, almost instantaneous".

Which to write: the paper is set from the faculty slides, so for a 2-mark "classify soft and near with an example" write: *Soft — deadlines preferred, occasional delay tolerable, latency ms–s (e.g. video streaming/VoIP). Near — processed with minimal lag of seconds (e.g. stock trading ticker, traffic monitoring)*. Quote the Table 1.1 latency numbers; avoid the word "instantaneous" and avoid stating which has higher tolerance. In a Part B answer, give the Table 1.1 table *and* mention the slide examples.

💡 **MNEMONIC — Hard–Soft–Near = H–S–N = "Microseconds, Milliseconds, Minutes"**

Hard **µs–ms** (no tolerance) → Soft **ms–s** (low) → Near **s–min** (high). Latency grows, tolerance grows.

1.2 Real-time vs streaming BOOK p.7 ↗

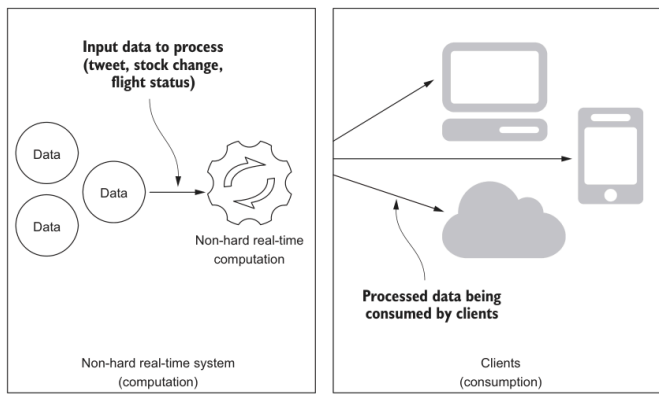


Figure 1.3 Real-time computation and consumption split apart

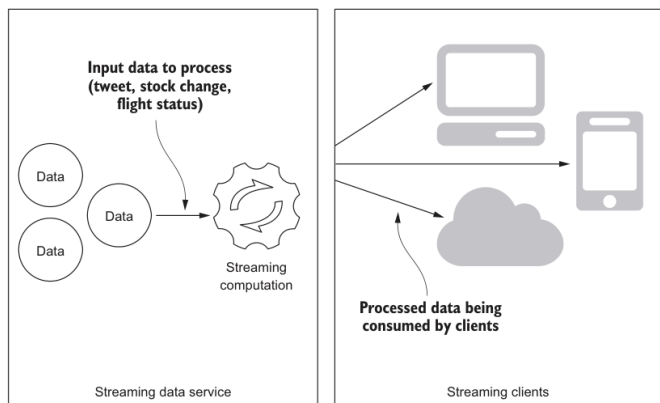


Figure 1.4 A first view of a streaming data system

Fig 1.3 Computation and consumption split apart (book p.7) Fig 1.4 A first view of a streaming data system (book p.8)

DEFINITION — Streaming data system
A **non-hard real-time service** whose computation runs continuously, with **clients that consume the data when they need it** (delays from network, app design, client not running). It is "neither soft nor near — it is **streaming**" / an **in-the-moment system**.

- Split the system in two: **computation** (non-hard real-time) and **consumption** (clients). Streaming = serve data at the moment the client needs it.
- Examples: **Twitter** (latest tweets on request, some seconds old, some hours old); **FlightAware** (latest status per airport/flight); **NASDAQ Real Time Quotes** (latest quote per stock).

Real-time system	Streaming data system
Judged by delay seen by consumer (soft/near/hard)	Judged by whether data is available when client asks
One word for both computation + consumption — ambiguous	Separates computation (non-hard RT) from consumption (clients pull/receive when ready)
Deadline-centric	Scalability, availability, continuous flow-centric

1.3 Need and characteristics of streaming data (faculty PPT slides 17–18)

IMPORTANT CIE-1 Dec'24 A2 · CIE-1 Nov'25 A2 (list any two)

- Some data **naturally comes as a never-ending stream** of events.
- Batch lets data pile up and processes at once; stream processes as it arrives, spreading the work over time.
- Data is so huge it may be **impossible to store**.
- Huge sources (transactions, activity, web visits) growing faster with IoT sensors.

Characteristics (slide 18): **Chronological significance** (order matters) · **Continuous flow** · **Uniqueness** · **Non-homogeneity** · **Imperfection**.

MNEMONIC — CCUNI — "Chrono, Continuous, Unique, Non-homogeneous, Imperfect"
Five characteristics of streaming data.

🔥 HOW TO WRITE IT — Part A "list two needs"

Write any two numbered points above in one line each — 1 mark each. Don't elaborate.

1.4 Stateless design (SEE Feb'26 Q1.1) **IMPORTANT** SEE Feb'26 1.1

- **Stateless** = the service keeps **no information about the client between requests**; every request carries everything needed. So a client can call **any** instance and instances can be added/removed freely. ([BOOK p.26 ↗](#) (book p.26: "we don't have any state information about the client... can easily add new instances"))
- *Live video analytics example*: each camera frame/chunk is sent to any analytics instance behind a load balancer, processed on its own and the result returned; no instance remembers earlier frames, so a crashed instance is simply replaced and auto-scaling works. (Example is ours; the idea is from p.26.)

2 · The architectural blueprint [BOOK p.9 ↗](#)

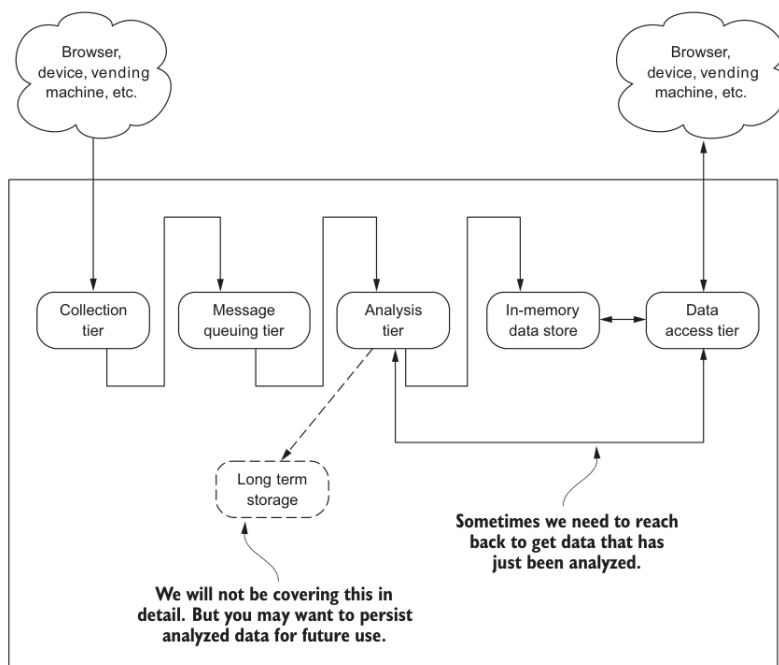


Figure 1.5 The streaming data architectural blueprint

Fig 1.5 The streaming data architectural blueprint (book p.9) — DRAW THIS

Tier	Job	Twitter example (book p.10)
Collection tier	Entry point: takes data from browsers/devices/vending machines etc.	User posts a tweet → collected by Twitter services
Message queuing tier	Decouples collection from analysis; buffers/transport	Collection and analysis may be in different data centres
Analysis tier	Stream processing / continuous queries on in-flight data	Identify the followers of a tweet
In-memory data store	Holds freshest results for fast access	Tweets a few seconds old
Data access tier	Delivers data to clients (API)	All Twitter clients connect to access the service
Long-term storage (optional, dashed)	Persist analysed data for later	Old tweets in a persistent store

- Tiers are **not rigid like LEGO pieces**, may be spread over several physical locations; they don't prescribe deployment. ([BOOK p.10 ↗](#))
- Message queuing tier is not stressed in Fig 1.7 but still plays a collection role in some cases.

🧠 MNEMONIC — C-M-A-I-D (+L)

Collection → Message queue → Analysis → In-memory store → Data access; Long-term storage hangs off the analysis tier.

Security for streaming systems ([BOOK p.10 ↗](#))

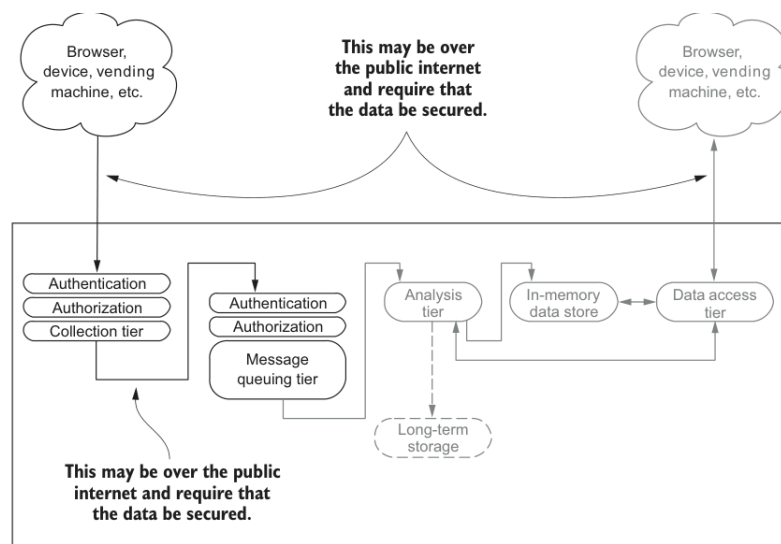


Figure 1.6 The architectural blueprint with security identified

Fig 1.6 Blueprint with security identified (book p.11)

- Security is **overlaid** on the blueprint, not a separate tier. Data travels over the **public internet** at the collection side and the data-access side ⇒ must be secured.
- Apply **Authentication** (who are you) and **Authorization** (what may you do) at the collection tier, the message queuing tier and the data access side. (Also encryption in transit/at rest — see Unit 2 Fig 3.14.)

3 · How do we scale?

BOOK p.11 ↗

IMPORTANT

CIE-1 Dec'24 A3 · Model QP 2b

	Vertical scaling (scale up)	Horizontal scaling (scale out)
Meaning	Add resources (CPU, RAM, disk) to the existing server	Add more servers and spread the load
Limit	Capped by maximum capacity of one machine	Practically unlimited; can shrink too (elasticity / auto-scaling)
Analogy (book)	Restaurant: more tables/chairs until max occupancy	Highway: add lanes; airport: open/close security lines
Example	Add memory to a collection node	Add nodes behind a load balancer

- Goal for every tier of a streaming system = **horizontal scaling**; vertical is used only to find an ideal resource config, then you scale out. (BOOK p.12 ↗)
- **Importance of scaling** (Model QP 2b, 6M): handle growing volume/velocity (millions of vehicles, 46,000 searches/s), avoid single-machine ceiling, keep latency low during spikes, elasticity saves cost, supports fault tolerance via redundancy.

4 · Common interaction patterns (collection tier)

BOOK p.15 ↗

Five patterns, whatever the protocol: **Request/response** · **Request/acknowledge** ·

Publish/subscribe · **One-way** · **Stream**. **IMPORTANT** SEE Feb'26 Q2a,Q2b,1.2 · CIE-1 Nov'25 A1,Q2b · CIE-1 Dec'24 Q2a,Q3a

🗨️ **MNEMONIC — R-R-A-P-O-S → "Really Rapid Acknowledgement, Pub, One, Stream"**

Request/Response · Request/Acknowledge · Pub/sub · One-way · Stream.

4.1 Request/response (BOOK p.16 ↗)

- Client sends request; service sends response over the **same connection**. Simplest, **synchronous**; client must wait. Use when an **immediate response** is needed (web browsing, Google search, buy a ticket).
- Three fixes for waiting: **half-async client side** (client sends async and does other work — all browsers), **half-async service side** (service delegates work then replies; more scalable), **full-async** (both).
- Example in book: real-time traffic & routing service for all vehicles — full-async variant chosen. (BOOK p.18 ↗)

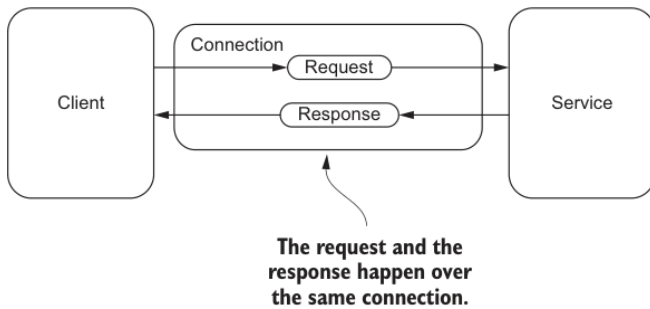


Figure 2.2 Basic request/response pattern

Fig 2.2 Basic request/response (p.16)

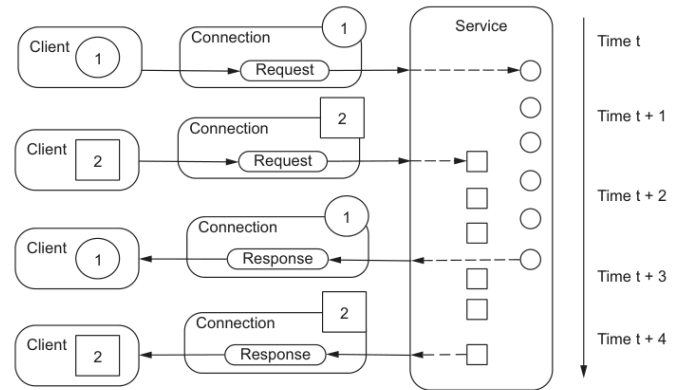


Figure 2.4 Service async request/response pattern

Fig 2.4 Service async request/response (p.17)

4.2 Request/acknowledge [BOOK p.19 ↗](#) **IMPORTANT** SEE Feb'26 1.2

- Like request/response but you only need **confirmation the request was received**, not the processed result. The acknowledgement carries a **token/ID** used in later requests (check status, get final result). Real life: order confirmation number.
- Book example: e-commerce **propensity-to-buy score** — first page visit returns a unique ID (ACK); later pages pass that ID to the propensity service. [BOOK p.19 ↗](#)

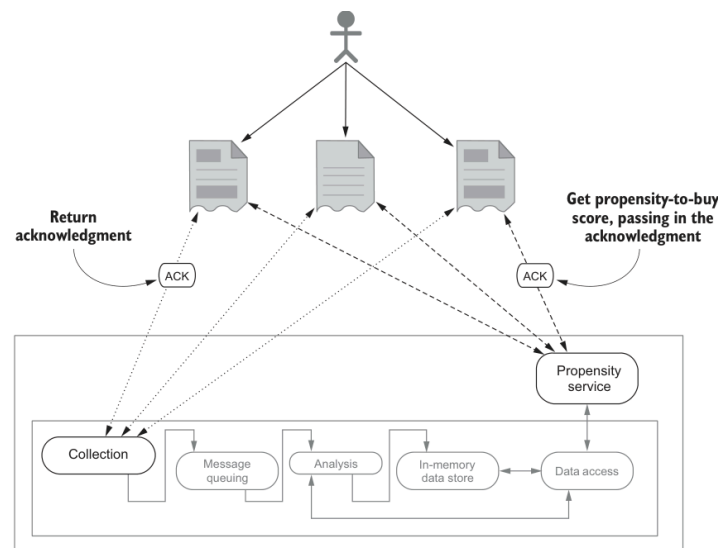


Figure 2.6 Visitor browsing while data is collected and their propensity-to-buy score is updated

Fig 2.6 Visitor browsing; ACK ID used to fetch propensity score (p.20)

🔥 HOW TO WRITE IT — SEE Feb'26 Q1.2 — warehouse sends an order update, only needs confirmation it was received

Request/acknowledge pattern. The warehouse system gets an acknowledgement token as soon as the message is received (no processed response needed), so it knows the message was not lost, can retry if no ACK arrives, and can use the token later to check status ⇒ improves reliability and keeps the sender non-blocking.

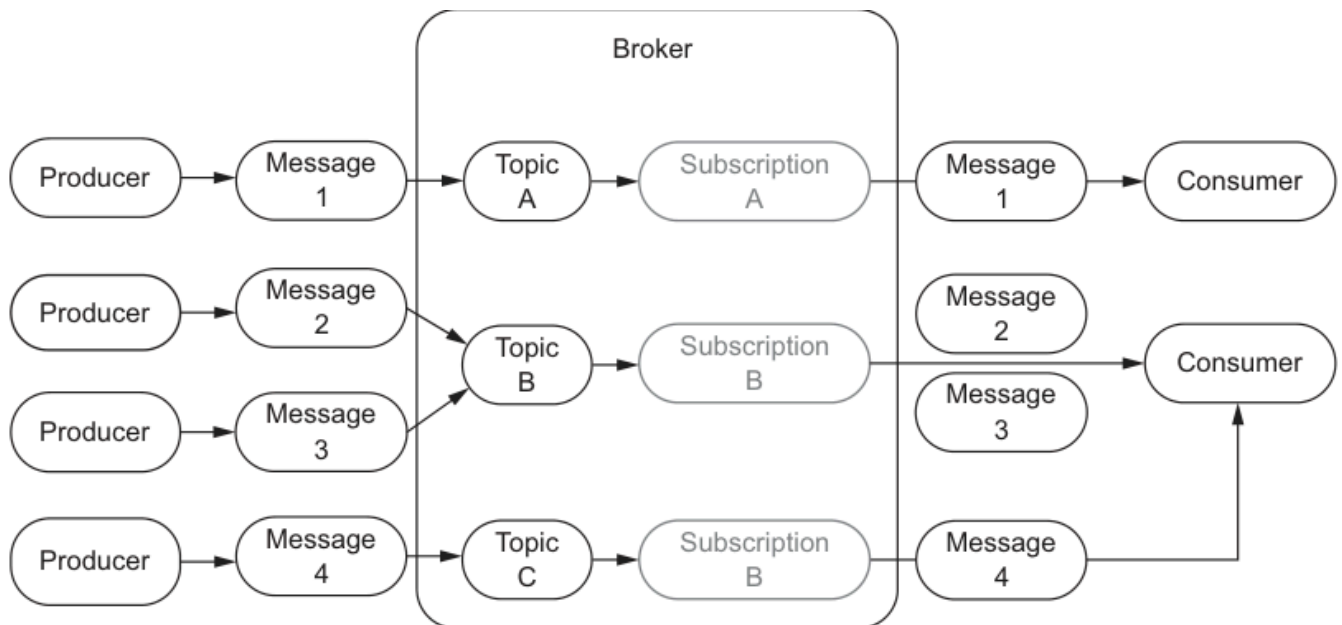


Figure 2.7 General data flow for the publish/subscribe message pattern

Fig 2.7 General data flow for the pub/sub pattern (book p.21) — DRAW THIS for Q2b

1. **Producer** publishes a message to a **broker**, usually into a **topic** (logical grouping).
2. Broker keeps topics and **subscriptions**; delivers the message to **all consumers subscribed** to that topic (some products push, others let the consumer pull).
3. Consumer reads it asynchronously. A producer need not subscribe; a subscriber need not produce.

Component (PPT slides 35–38)	Role
Publisher	Creates messages, classifies them by topic; doesn't know subscribers
Subscriber	Declares interest in topics; doesn't know publishers
Topic	Named channel/category of messages
Message broker	Intermediary: routes, can add persistence, scalability, reliability
Message	Unit of data (text, JSON, binary)
Subscription	Link subscriber→topic; may carry delivery guarantees (at-most/at-least-once) and ack rules

- **Benefit:** **decouples sender from consumer** (time, space, identity) — vital when scaled from 4 cars to all cars in the US. Fully **asynchronous**.
- **Example (book Fig 2.8):** cars publish "current traffic" to a *Traffic* topic; a car with a traffic subscription consumes it. [BOOK p.22](#)

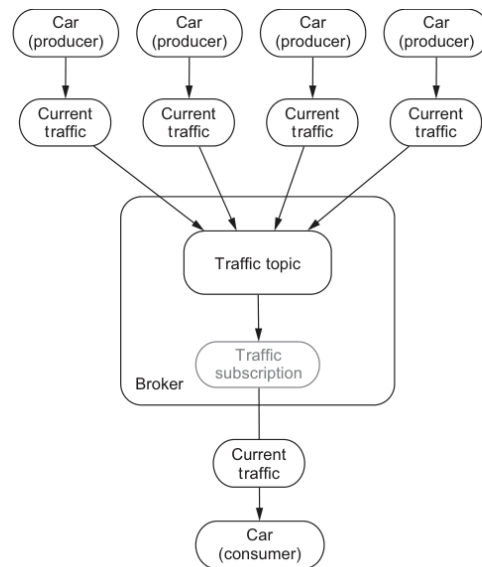


Figure 2.8 Current traffic publish/subscribe message pattern

Fig 2.8 Current traffic pub/sub (p.22)

🔥 HOW TO WRITE IT — Q2b 8 marks

Definition (1) → labelled Fig 2.7 (3) → 6 flow steps (2) → components table + benefit (1) → example (1).

4.4 One-way pattern ("fire and forget") [BOOK p.22 ↗](#)

- Client sends, service sends **no response**; client doesn't even know whether it arrived. Suits clients with no resources/need to process a reply, and where **losing some data is tolerable** for simplicity, low resource use and speed.
- Examples: server sending CPU/memory stats every 10 s; **NFL RFID tags** sending 25 samples/s — losing 5 doesn't change the analysis; weather collector sending periodic updates (CIE-1 Nov'25 A1b).

4.5 Stream pattern [BOOK p.23 ↗](#) **IMPORTANT** SEE Feb'26 Q2a (8M) · CIE-1 Dec'24 Q3a "ingesting a stream and producing another stream" (5M) · CIE-1 Nov'25 A1a

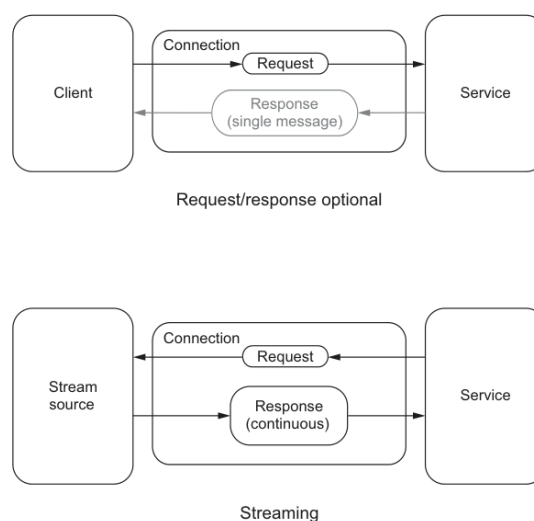


Figure 2.9 Comparing the request/response patterns to the stream pattern

Fig 2.9 Request/response optional vs streaming (p.23)

- The pattern is **flipped**: the service becomes the client. The collection tier **connects to a stream source and pulls data in**; one request → **continuous flow** of responses (vs 0 or 1 response).
- Request/response optional: an *external client pushes* messages to you. Stream: *you connect and consume* (e.g. connect to Twitter for sentiment analysis; Meetup RSVP JSON stream over one long-lived HTTP connection).
- **Ingest a stream, produce another stream**: build an analysis system that consumes a public stream and publishes a new stream derived from your analysis.
- Characteristics (PPT 43): asynchronous, multiple data chunks, non-blocking. Use for live updates, chat, video streaming; request/response for one-off API calls.

 HOW TO WRITE IT — SEE Feb'26 Q2a "streaming pattern with example" (8M)

Definition + flip-of-roles diagram (Fig 2.9 bottom) + 4 differences vs request/response + one example (Meetup RSVP stream / Twitter sentiment: collection tier connects, consumes JSON events, pushes to message queue) + scaling note (buffering layer, §5).

4.6 Compare patterns (Part B "describe and compare") **IMPORTANT** CIE-1 Nov'25 Q2b (5M)

Pattern	Response?	Sync?	Typical use / example	Reliability
Request/response	Yes, full	Sync (or half/full-async)	Web, REST, search	Client knows result
Request/acknowledge	ACK token only	Async	Order/job submit, propensity score	Receipt confirmed
Pub/sub	None to publisher	Async	Traffic topic, Kafka, Slack channels	Via broker guarantees
One-way	None	Async	Telemetry, RFID, logs	Lossy by design
Stream	Continuous	Async, non-blocking	Tweets, clickstream, live video	Needs buffering + logging

Choose (CIE-1 Nov'25 A1, solution): e-commerce clickstream → **Stream pattern** (continuous events); weather collector sending periodic updates with no reply → **One-way pattern**.

5 · Scaling the interaction patterns **BOOK p.25 ↗**

5.1 Request/response-optional patterns **BOOK p.26 ↗**

- Scale **horizontally** because the pattern is **stateless**: (1) a client can connect to **any** instance, (2) new instances can be added **without changing** running ones. Cloud **auto-scaling** adds/removes instances with demand.
- Scale of example: ~253 million US vehicles, 2.966 trillion miles (2012) ⇒ thousands–millions of simultaneous requests. A **load balancer** routes to collection nodes, which pass data to the message queuing tier.

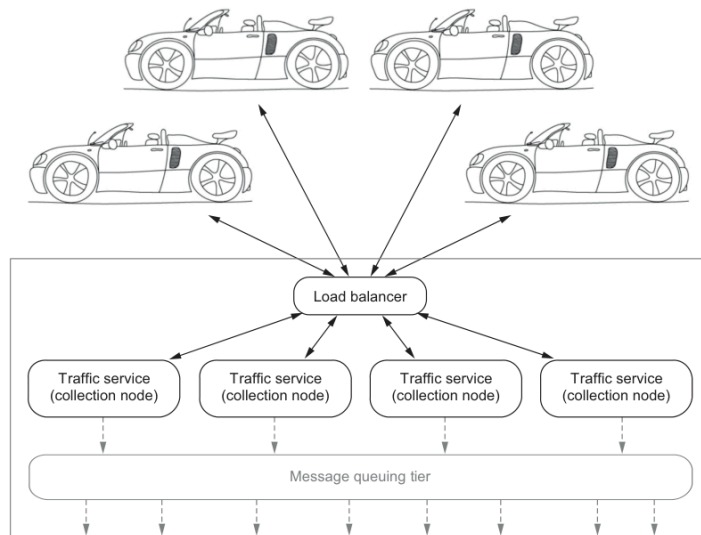


Figure 2.10 Vehicle and traffic service with a load balancer

Fig 2.10 Vehicle and traffic service with a load balancer (p.26)

5.2 Stream pattern [BOOK p.27 ↗](#)

- Problem: a stream connection is **direct and persistent** to *one* collection node $\Rightarrow$ the other nodes **sit idle** (46,000 Google search events/s example).
- Solutions (not exclusive): **(a) scale up** the consuming node (limited by hardware) and **(b) add a buffering layer**. Key rule: **no business logic** on messages read from the stream; push them to the buffer fast; a second set of collection nodes (that do the business logic) reads from the buffer and is **horizontally scalable**. [BOOK p.28 ↗](#)

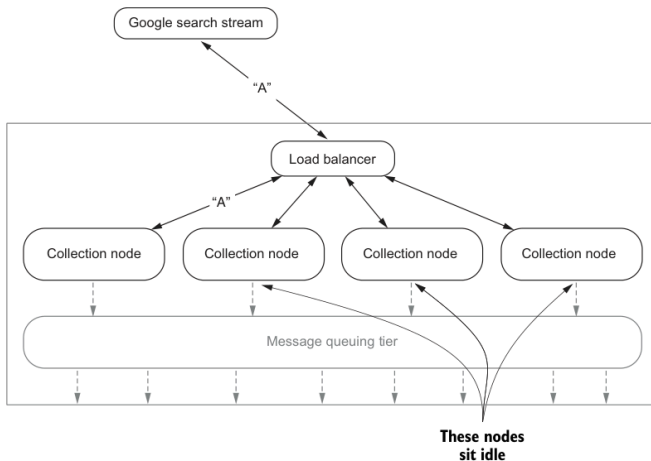


Figure 2.11 Search stream with direct connection to single collection node

Fig 2.11 Direct connection – idle nodes (p.27)

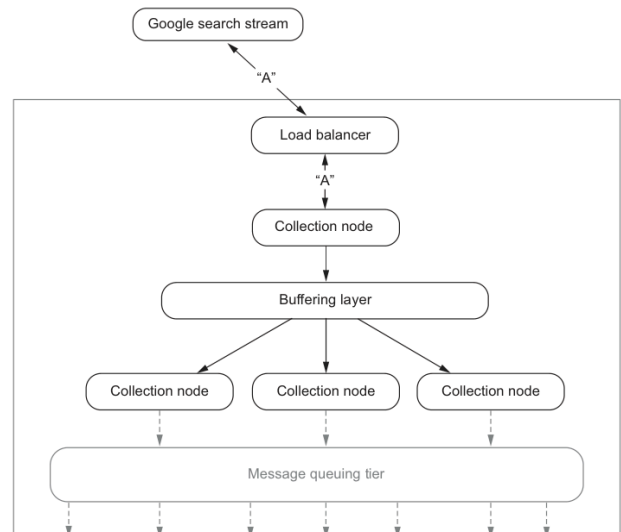


Figure 2.12 Collection tier with buffering layer in place

Fig 2.12 Buffering layer in place (p.28)

6 · Fault tolerance in the collection tier [BOOK p.28 ↗](#)

- Collection nodes **will** fail (bug, 3rd-party software, hardware). Redundancy alone is not enough because the **message from a client may not be reproducible** — the client can't resend. Goal: **mask failures, don't lose data, recover as if the crash never happened.**
- Two approaches: **Checkpointing** and **Logging**. [BOOK p.29 ↗](#)

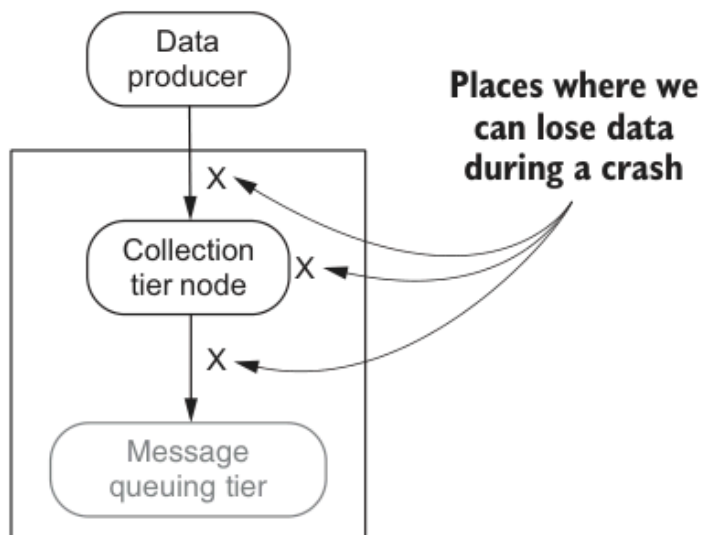


Figure 2.13 Collection scenario with our data-loss potential identified

Fig 2.13 Places where data can be lost during a crash (p.29)

6.1 Checkpointing **IMPORTANT** Model QP 1.2 · CIE-1 Nov'25 Q1a

- Regularly save a **global snapshot** of the **whole system** (collection → data access), not just one tier, to durable storage. On recovery, restore the last snapshot.
- Potential for data loss:** only recoverable up to the last snapshot; messages processed afterwards are lost. Analogy: MS Word/Google Docs autosave.
- Hard to take a consistent global snapshot across many layers/technologies ⇒ book calls it **a bad choice for a streaming system** (OK for building HDFS/NoSQL). [BOOK p.30 ↗](#)

6.2 Logging protocols

- Aim: avoid checkpoint expense/complexity and **recover up to the last message received**. Idea: **if a message can be replayed, the system reaches a consistent state without a global snapshot**. Each tier independently logs the messages it receives and replays after a crash. [BOOK p.30 ↗](#)

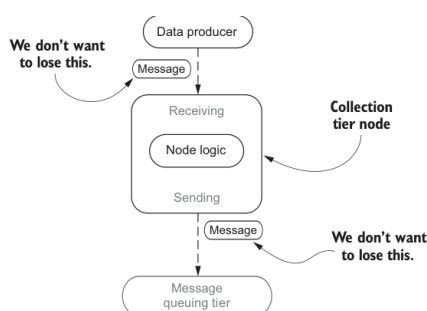


Figure 2.14 High-level overview of receiver-based and sender-based message logging

Fig 2.14 RBML protects incoming, SBML protects outgoing (p.31)

	RBML — receiver-based BOOK p.31 ↗	SBML — sender-based BOOK p.33 ↗	HML — hybrid BOOK p.34 ↗
What is logged	Every received message , synchronously to stable storage, before any processing (raw input)	Every outgoing message after node logic, before sending to next tier	RBML + SBML combined
Protects against	Crash of this node while handling the message ("protect ourselves from ourselves")	Next tier crashing / network interruption	Both
Recovery	Take node out of load-balancer rotation → RBML logger replays unprocessed messages through node logic → put back in rotation	Replay logged messages; need to know if next tier already got them ⇒ ACK from queuing tier, then delete/mark the logged message	Same protection, less overhead
Cost	Extra write per message (throughput hit)	Extra write + ACK handling	One consolidated stable storage + asynchronous writes ; reported 43% overhead reduction vs RBML (Meyer et al.)

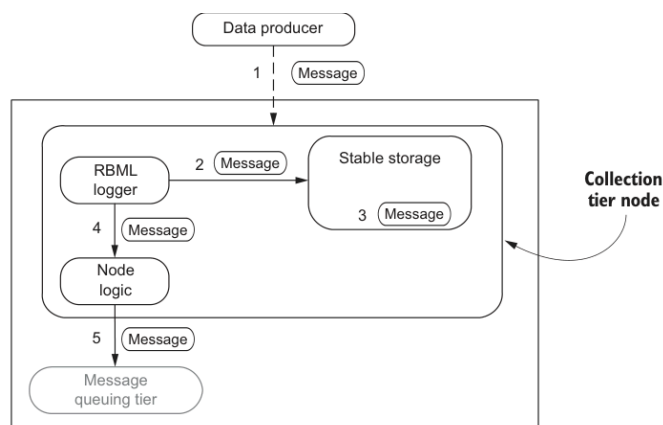


Figure 2.15 RBML implemented for the simple collection node showing the happy path

Fig 2.15 RBML happy path: 1 message → 2 logger → 3 stable storage → 4 node logic → 5 queue (p.32)

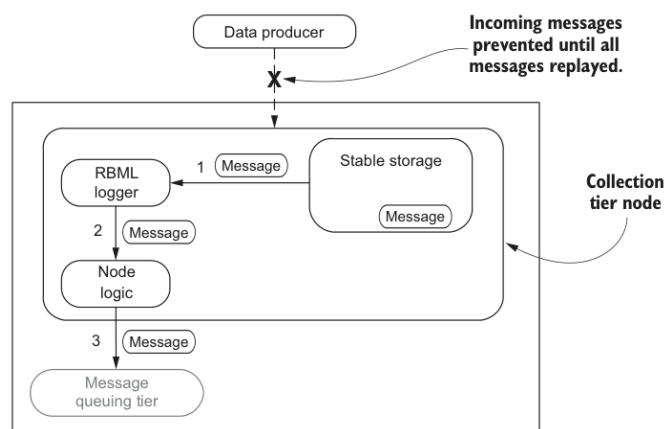


Figure 2.16 The recovery data flow for RBML

Fig 2.16 RBML recovery (p.32)

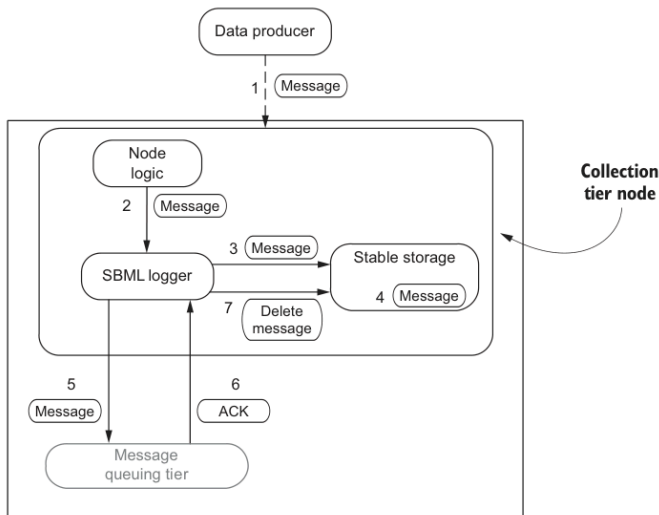


Figure 2.17 The normal execution data flow for SBML

Fig 2.17 SBML normal flow, steps 1–7 incl. ACK + delete (p.33)

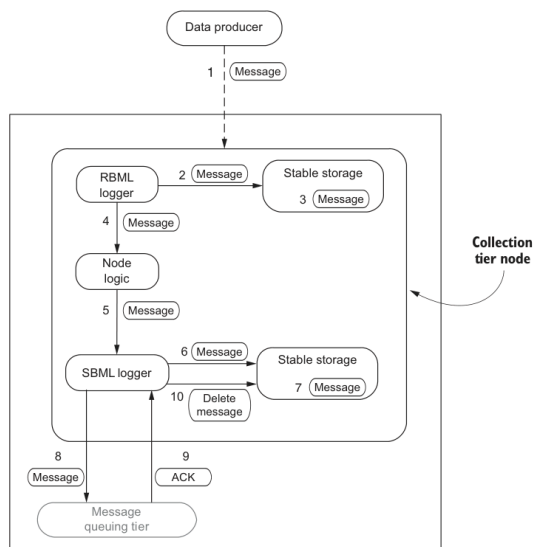


Figure 2.19 RBML and SBML together in the collection tier node

Fig 2.19 RBML + SBML together – 10 steps (p.35)

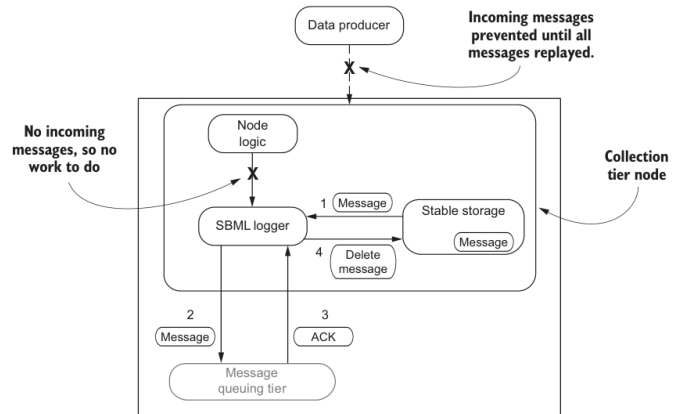


Figure 2.18 Recovery data flow for SBML

Fig 2.18 SBML recovery (p.34)

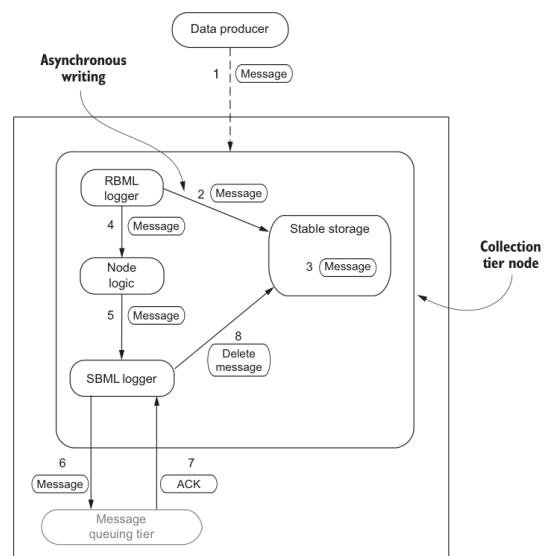


Figure 2.20 HML sample data flow

Fig 2.20 HML sample flow, asynchronous writing (p.36)

🧠 **MNEMONIC** — RBML = "Received → Back it up first"; SBML = "Sent → Back it up first"

RBML logs at the **front door**, SBML logs at the **back door**.

⚠️ **CONFLICT / TRAP** — Which logging technique? — read the scenario carefully

CIE-1 Nov'25 A3 (node loses messages after a *power failure*) — official solution: **RBML**: it saves every received message before processing, so a crashing collector can replay all of them.

SEE Feb'26 1.3 (a *sensor gateway crashes* but all upstream sensor nodes stay up **and can resend**) — no scheme seen. The upstream nodes hold the messages, so the matching technique is **SBML** (senders log before sending and replay to the crashed receiver). This is our reading of the question; if you prefer to hedge, add "RBML would also work at the gateway if senders could not resend".

Rule: **the node itself may lose input** → **RBML**; **the next tier/receiver crashes and the sender holds copies** → **SBML**; **want both with low overhead** → **HML**.

⚠ CONFLICT / TRAP — Checkpointing — book vs solution scheme

The textbook (p.30) says checkpointing is a **bad fit** for streaming systems (global snapshot is hard, data loss after last snapshot). The CIE-1 Nov'25 solution (from the faculty slides) says "*checkpointing and logging jointly ensure reliable recovery: checkpoint restores state, log replays lost data*" and cites Flink checkpoints to S3. **Write both:** state the two characteristics (global snapshot, possible data loss) and then, if the question says "jointly", describe *restore last checkpoint + replay logged messages after it*. Slide 54 also lists these two characteristics.

6.3 A dose of reality (BOOK p.37 ↗)

Book anecdote: dashboards over WebSockets kept up with only ~60% of the stream and users still loved them — don't over-engineer; reflect on "must-have" vs reality.

7 · Quick revision — Unit 1

Three classes of real-time system?	Hard, soft, near.
Hard real-time example?	Pacemaker, anti-lock brakes (μ s–ms, failure = disaster).
Streaming system in one line?	Non-hard real-time service whose clients consume data when needed.
Five interaction patterns?	Req/resp, req/ack, pub/sub, one-way, stream.
Which pattern is "fire and forget"?	One-way.
Which pattern returns a token?	Request/acknowledge.
Stream pattern flips what?	Service becomes the client — collection tier pulls from the stream source.
Why is a buffering layer needed for stream pattern?	Direct connection leaves other collection nodes idle; buffer decouples and lets nodes scale out.
Why scale horizontally?	Stateless, no ceiling, auto-scaling.
Checkpoint = ?	Periodic global snapshot; loses data after last snapshot.
RBML / SBML logs when?	RBML: on receipt, before processing. SBML: before sending to next tier.
HML advantage?	Same safety, less overhead (one storage, async writes, –43%).

8 · Worked answer — streaming architecture for a scenario

IMPORTANT CIE-1 Dec'24 Q1a (10M) · CIE-1 Nov'25 Q1a (10M) · Model QP Q2a (10M)

Scenario (CIE-1 Dec'24): Vehicle tracking — monitor location, speed, fuel of a fleet, alert on anomalies.

1. **Collection tier:** on-board GPS/OBD units send location, speed, fuel every few seconds (**one-way/stream pattern**, behind a **load balancer**, stateless nodes; RBML logging so no reading is lost). Authenticated (TLS + device credentials).
2. **Message queuing tier:** Kafka-like broker with a *vehicle-events* topic, **durable**, partitioned by vehicle ID; decouples spikes from analysis.

3. **Analysis tier:** stream processor (Storm/Flink/Spark) runs **continuous queries**: speed > limit, fuel drop rate, geofence exit → raises anomaly alerts; sliding window for averages.
4. **In-memory data store:** latest position/status per vehicle (Redis/IMDB) for the live map.
5. **Data access tier:** API/WebSocket/pub-sub pushing live location + alerts to dashboards and the driver app.
6. **Long-term storage:** trips and alert history in HDFS/S3 for reports.
7. **Cross-cutting:** security (authN/authZ each tier), horizontal scaling per tier, fault tolerance (replicated brokers, logging).

For the IoT environmental network (CIE-1 Nov'25 Q1a, solution): Sensors → edge gateways → broker (Kafka/Pulsar) → Flink/Spark → hot store (Redis)/cold store (S3/HDFS) → dashboards/alerts; pattern = **pub/sub + stream**; scaling = **horizontal** (vertical only for stateful nodes); fault tolerance = replication + checkpoints + message logging; security = TLS/mTLS, device auth, RBAC.